

Enhancing Echo State Networks with Gradient-based Explainability Methods

Francesco Spinnato*, Andrea Cossu, Riccardo Guidotti, Andrea Ceni, Claudio Gallicchio, Davide Bacciu

*francesco.spinnato@di.unipi.it
University of Pisa

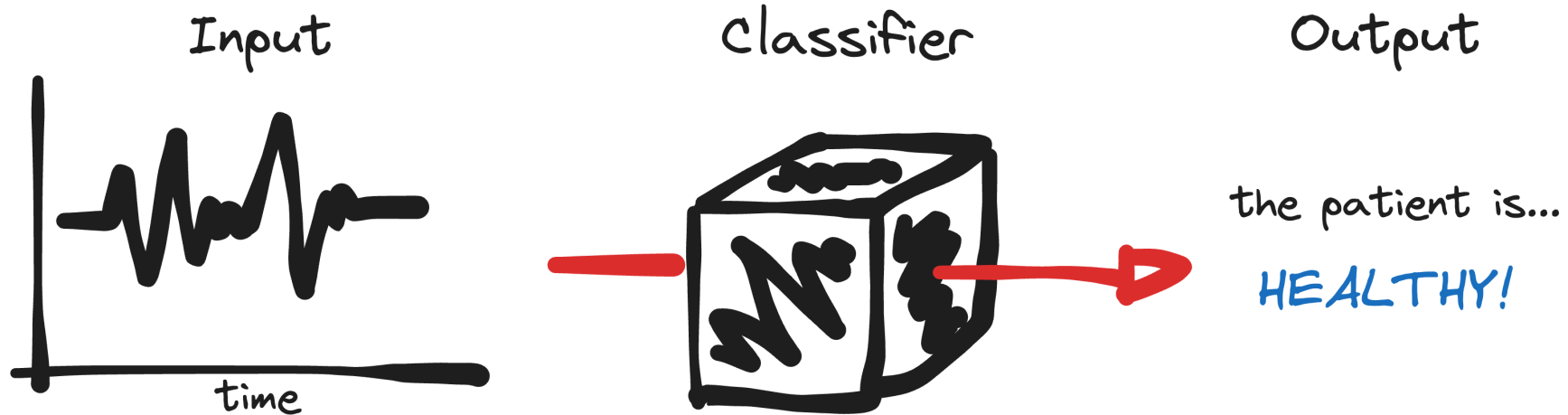
ESANN 2024



Binary Time Series Classification

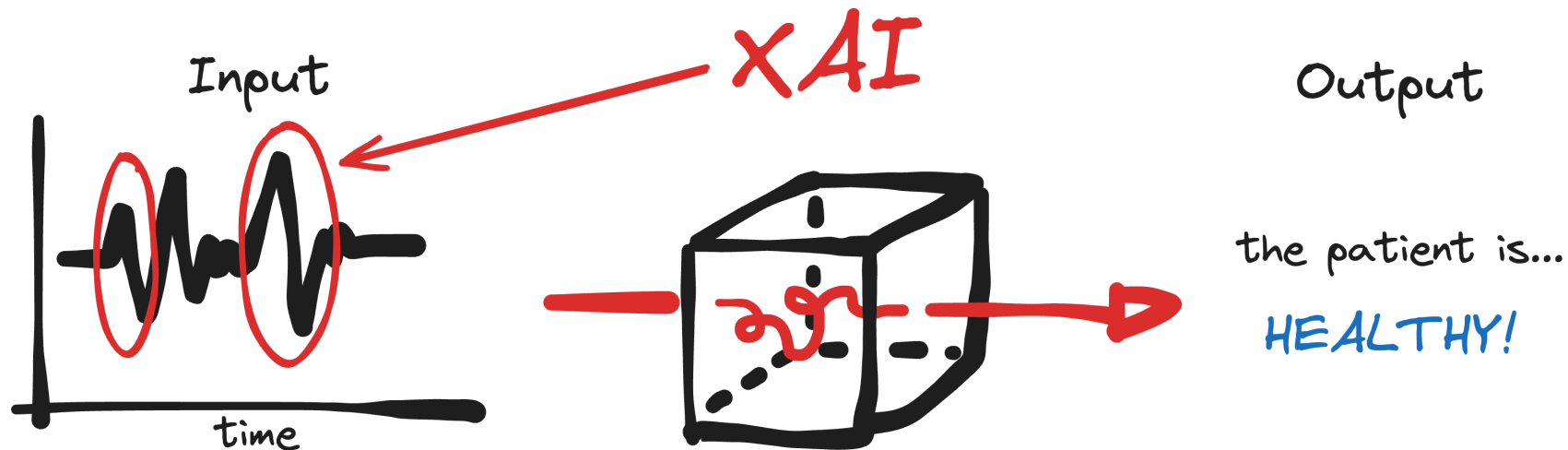
Training a model f that maps a time series, $\mathbf{x}$, to a label, $\hat{y} = f(\mathbf{x})$.

- univariate time series, $\mathbf{x} = [x_1, \dots, x_T]$, of length T .
- in a time series dataset $\mathbf{X} \in \mathbb{R}^{N \times T}$, with labels, $\mathbf{y} \in \{0, 1\}^N$;



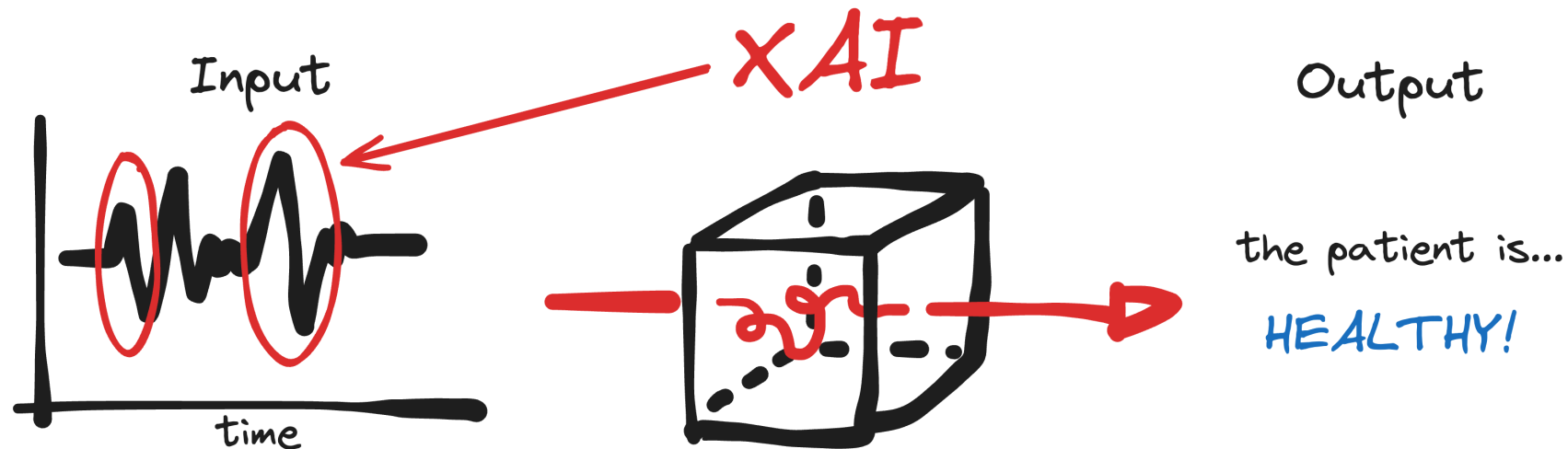
XAI for Time Series Classification

Explainable AI (XAI) is mostly used to understand the **relationship between the input and the output**, for example, by highlighting the most important observations in the input.



Enhancing Models with XAI Methods

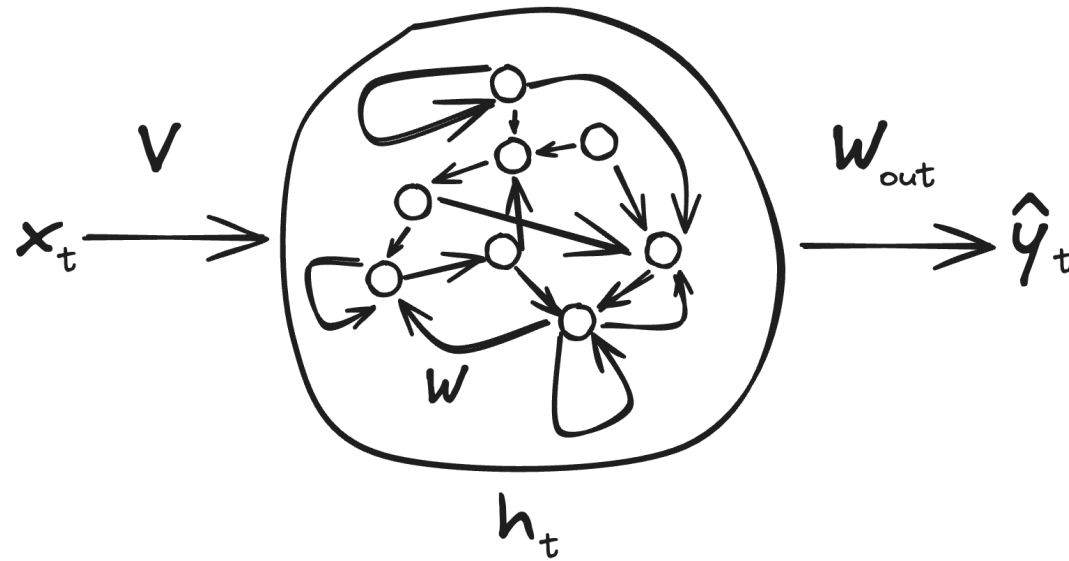
In this work we try to use XAI approaches to improve the performance of models instead of explaining them.



Echo State Networks

As models, we use **Echo State Networks (ESNs)**.

They are RNNs characterized by a fixed recurrent component called **reservoir** and a trained output layer called **readout**.



Echo State Networks - Hidden State

The ESN updates its hidden state, $\mathbf{h}_t$, according to:

$$\mathbf{h}_t = \sigma(W\mathbf{h}_{t-1} + Vx_t + \mathbf{b})$$

- σ : nonlinear activation function;
- x_t : input at time t ;
- W : recurrent matrix (initialized randomly and scaled by ρ);
- V : input matrix (scaled by ν).

Echo State Networks - Readout

The readout is the only trained part of the network and **can be trained in closed form**. It takes as input the hidden states and outputs a class:

$$\hat{y} = \sigma(W_{\text{out}}\mathbf{h}' + \mathbf{b}_{\text{out}})$$

- $W_{\text{out}}, \mathbf{b}_{\text{out}}$: weights and bias of the readout;
- σ : sigmoid activation function.

Usually, the input of the readout is either the **last** hidden state, ($\mathbf{h}' = \mathbf{h}_T$) or a simple **average** of the hidden states ($\mathbf{h}' = \bar{\mathbf{h}}$).

We propose weighting the reservoir output, based on the importance of each observation in $\mathbf{x}$.

$$\mathbf{h}' = \sum_{t=1}^T \phi_t \mathbf{h}_t \quad \text{with} \quad \sum_{t=1}^T \phi_t = 1$$

Enhancing Echo State Networks

$$\mathbf{h}' = \sum_{t=1}^T \phi_t \mathbf{h}_t \quad \text{with} \quad \sum_{t=1}^T \phi_t = 1$$

This formulation generalizes the standard cases:

- **Last hidden state:** $\phi = [0, 0, \dots, 0, 1]$
- **Average of hidden states:** $\phi = \left[\frac{1}{T}, \frac{1}{T}, \dots, \frac{1}{T} \right]$

Weighting the Hidden States

We use model-specific, gradient-based XAI methods to **infer the importance of points** in the time series.

The foundation of these approaches is the **analysis of the gradient** of the network's output w.r.t. an input instance, as it informs about the sensitivity of the output to the input features.

These techniques can **highlight the importance of each observation** in the time series with a saliency vector:

$$\phi = [\phi_1, \dots, \phi_T]$$

Weighting the Hidden States

- ***Gradient***: computes the gradient of the output w.r.t. each observation of the input time series:

$$\phi = \left[\frac{\partial \hat{y}}{\partial x_1}, \dots, \frac{\partial \hat{y}}{\partial x_T} \right].$$

Weighting the Hidden States

- **Gradient**: computes the gradient of the output w.r.t. each observation of the input time series:

$$\phi = \left[\frac{\partial \hat{y}}{\partial x_1}, \dots, \frac{\partial \hat{y}}{\partial x_T} \right].$$

- **Gradient*Input***: multiplies the gradient with the input: $\phi = \left[\frac{\partial \hat{y}}{\partial x_1} x_1, \dots, \frac{\partial \hat{y}}{\partial x_T} x_T \right].$

*Shrikumar, Avanti, et al. "Not just a black box: Learning important features through propagating activation differences." arXiv preprint arXiv:1605.01713 (2016).

Weighting the Hidden States

- *Gradient*: computes the gradient of the output w.r.t. each observation of the input time series:

$$\phi = \left[\frac{\partial \hat{y}}{\partial x_1}, \dots, \frac{\partial \hat{y}}{\partial x_T} \right].$$

- *Gradient*Input*: multiplies the gradient with the input: $\phi = \left[\frac{\partial \hat{y}}{\partial x_1} x_1, \dots, \frac{\partial \hat{y}}{\partial x_T} x_T \right].$

- *GradientSHAP**: uses a baseline vector to infer the feature importance:

$$\phi = \left[\frac{\partial \hat{y}}{\partial x_1} (x_1 - z_1), \dots, \frac{\partial \hat{y}}{\partial x_T} (x_T - z_T) \right].$$

*Scott, M., and Lee Su-In. "A unified approach to interpreting model predictions." Advances in neural information processing systems 30 (2017): 4765-4774.

Weighting the Hidden States - Normalizing the Saliency

We take its absolute value and normalize the saliency vector so that it sums to one:

$$\phi = \frac{[|\phi_1|, \dots, |\phi_T|]}{\sum_{i=1}^T |\phi_i|}$$

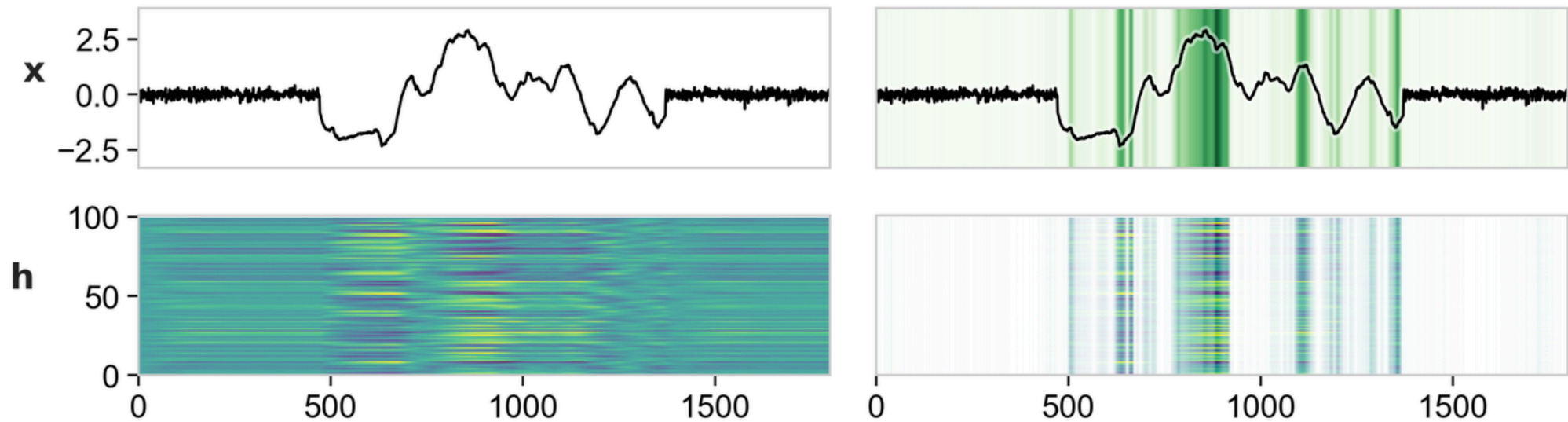
Weighting the Hidden States - Step by Step

To maintain efficiency, the readout is retrained only once i.e.:

1. We first train the readout of the ESN on $\bar{\mathbf{h}}$;
2. We compute the hidden state importance using an XAI method;
3. We scale the hidden states using the saliency vector;
4. We re-train the readout of the ESN.

Weighting the Hidden States - An Example

A time series from the *WormsTwoClass* dataset with some Gaussian noise.



Experiments

We leave the parameters of the ESN fixed at standard values: 100 units, $\rho = 0.99$, $\nu = 1$, and leak rate of 0.01.

We weight the hidden states using:

- *last* hidden state (last);
- *average* of hidden states (avg);
- *random* weights (rnd);
- *Gradient* (grad);
- *Gradient*Input* (grad*inp);
- *GradientSHAP* (gradshap).

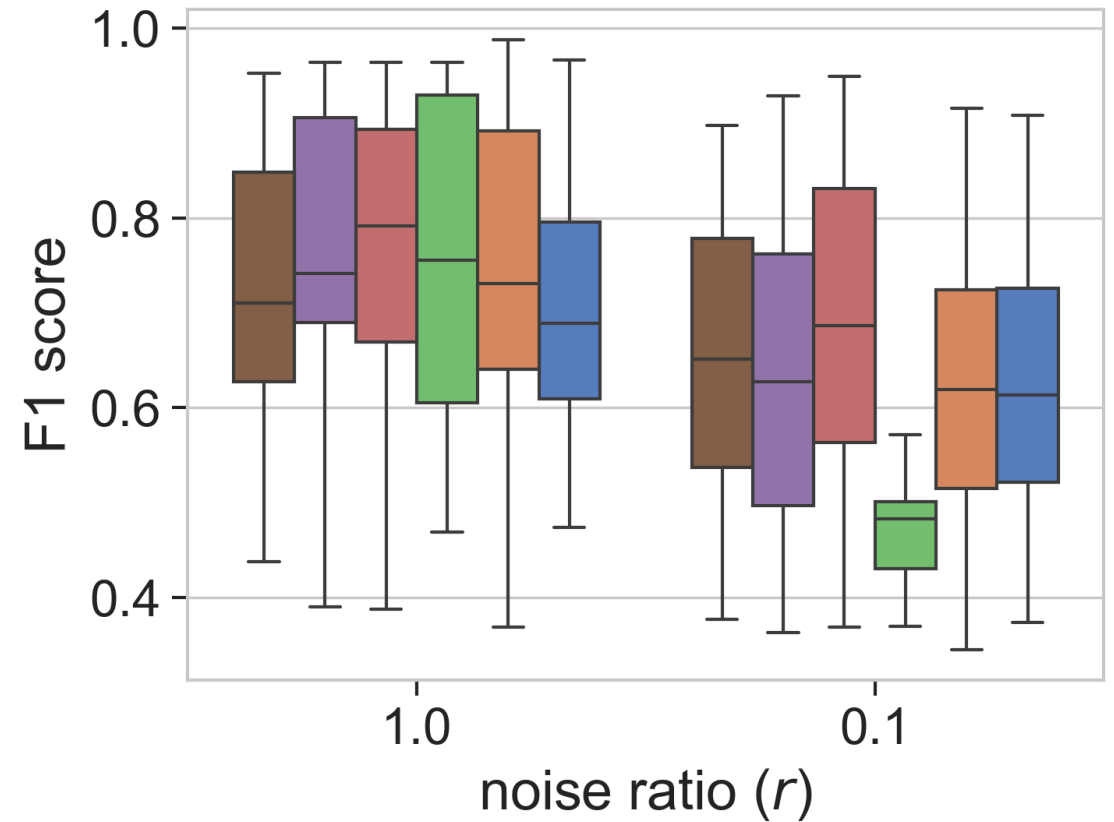
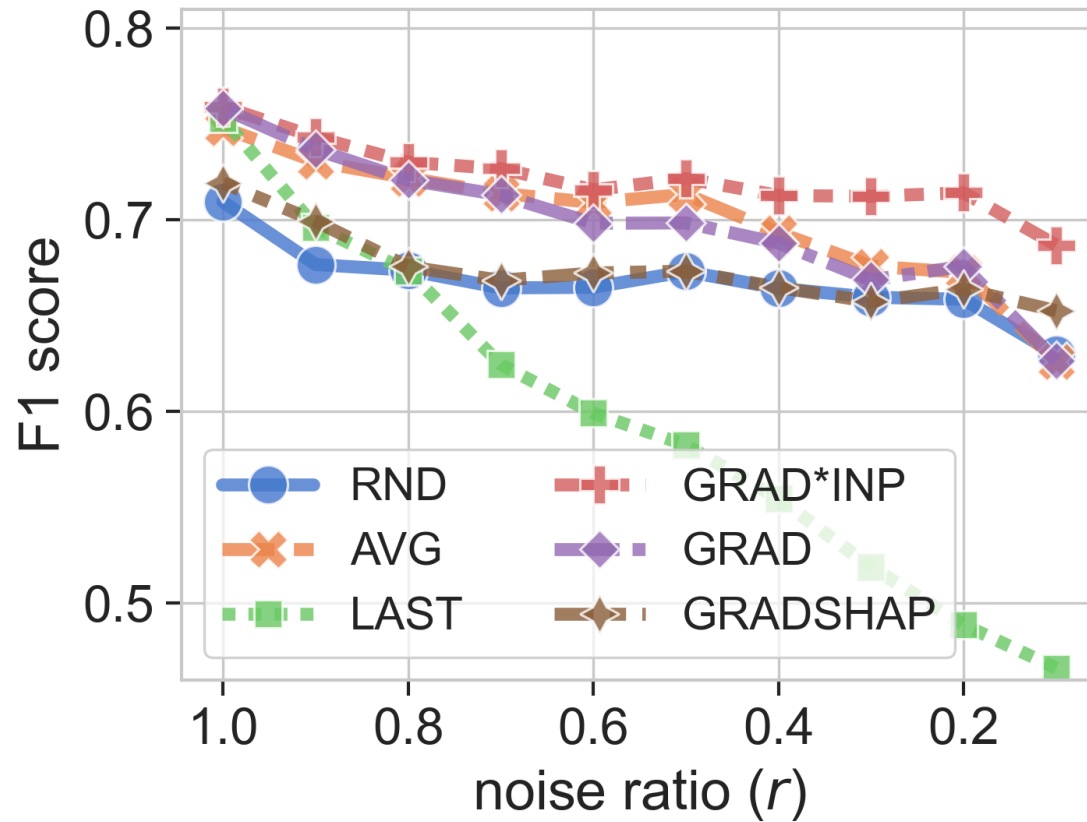
We benchmark our approach on 30 univariate binary classification datasets from the **UCR time series classification repository**^{*}, with increasingly more Gaussian noise.

The intuition is that the more *diluted* the hidden state is, the better gradient-based approaches will work.

As a base model to compute gradients we use *avg*.

^{*}Dau, Hoang Anh, et al. "The UCR time series archive." IEEE/CAA Journal of Automatica Sinica 6.6 (2019)

Results



Limitations and Future Works

For now, this approach **works only on binary classification**.

The multiclass problem is not a naive endeavor, as it requires considering the **gradient of multiple output nodes** with respect to the time series input.

Tests on **other RNN architectures** could be interesting.

Time Series Regression (and Forecasting) could benefit from this, but usually the last hidden state is the most important one.

THANK YOU FOR THE ATTENTION!

 https://github.com/fspinna/xai_enhanced_esn

EMERGE



 SOBIDATA
RESEARCH INFRASTRUCTURE



Future
Artificial
Intelligence
Research

HUMANE



AI NET